

CleanWater Access Tracker

Full Project Brief — Hackathon Build Plan (Solo, 12-hour build: 9 PM – 9 AM)

Project Summary

In many parts of Pakistan, communities struggle to track and report the condition of local water sources (contamination, shortage, pump breakdowns). This project is a working prototype that helps a local NGO or authority monitor this problem using real or realistic data. The tool takes messy, raw field data (CSV) and turns it into a clear, prioritized action list — helping a field worker decide which area needs a water tanker first, or which hand pumps need urgent repair.

Goal: the tool should be realistic and useful enough that it could genuinely be handed to an NGO or local council, not just something that runs without errors.

1. Mandatory Requirements (Already Implemented / Must Be Implemented)

Dataset (CSV → Database) — A realistic sample dataset of water sources with columns: area/locality name, district, source type (handpump, tap, tube well, filtration plant), water quality status (Safe / Unsafe / Needs Inspection), last inspection date, population served, and complaint count. ~80-120 rows, intentionally including missing values and inconsistent text entries. Loaded once and persisted into a real database rather than re-read from CSV on every request.

Data Cleaning (Pandas) — Handle missing or inconsistent entries — missing inspection dates, blank fields, inconsistent casing/spelling in status fields (e.g. 'unsafe' vs 'Unsafe' vs 'UNSAFE') — before it is written to the database.

NumPy Statistics (minimum 2) — e.g. average population affected per contaminated source, and the percentage of sources overdue for inspection.

Matplotlib Visualizations (minimum 3) — Areas ranked by urgency (population affected vs. water quality status); trend of complaints over time; distribution of source status (Safe / Unsafe / Needs Inspection) across areas.

Priority Action Report — An automatically generated plain-text report naming the top 3–5 locations needing the most urgent attention, based on a custom scoring formula combining population affected, days since last inspection, and complaint count.

Graphical User Interface (GUI) — The entire tool wrapped in a GUI so a non-technical NGO field worker can load data and see insights without touching code. Built as a modern React.js frontend (the brief allows any GUI framework) connected to a Python backend that performs all data processing, so the required libraries (Pandas, NumPy, Matplotlib) run exactly where they're needed.

2. Priority Scoring Logic (Core Algorithm)

Each water source is scored using a weighted formula so the ranking is explainable and defensible:

$$\text{priority_score} = (0.40 \times \text{normalized_population_served}) + (0.35 \times \text{normalized_days_since_inspection}) + (0.25 \times \text{normalized_complaint_count})$$

Multipliers applied on top of the base score: $\times 1.5$ if `water_quality_status = 'Unsafe'`, $\times 1.2$ if `water_quality_status = 'Needs Inspection'`. All numeric fields are min-max normalized (0–1) using NumPy before scoring. The top 3–5 highest-scoring areas populate the Priority Action Report.

3. Tech Stack & Architecture

Layer	Technology	Purpose
Frontend / GUI	React.js (Vite) + Tailwind CSS + Axios	Modern, non-technical-friendly interface
Backend / API	Python + FastAPI (centralized, layered)	Serves cleaned data, stats, charts, and report via REST endpoints
Database	SQLite (via SQLAlchemy ORM) for the hackathon; PostgreSQL for prod	Persistent structured storage. Instead of re-reading CSV each time
Data processing	Pandas, NumPy (inside backend, one-time ingestion, statistics and stats)	Core data processing and analysis libraries run here
Visualization (required)	Matplotlib (backend, returns chart images/baselines)	Satisfies the 3+ chart requirement
Security layer	AES payload encryption (backend: cryptography; Frontend: axios interceptors) + CORS	Protects API endpoints and responses — See Section 6
Optional AI layer	Google Gemini free API (or template-based NLP fallback)	AI for language summary generation
Version control	GitHub (public repo, monorepo: /frontend + /backend)	Code submission
Deployment (bonus)	Vercel (frontend) + Render / Railway (backend)	Free hosting, two linked deployments

Architecture flow: React frontend loads/uploads the CSV once and calls FastAPI endpoints (/upload, /stats, /charts, /priority-report). FastAPI runs Pandas cleaning on ingestion, stores structured records in the database, then serves NumPy calculations, Matplotlib chart images, and the priority report from the database on each request — not by reprocessing the raw CSV every time. CORS is restricted to the deployed frontend's domain so only the intended client can call the API.

Trade-off note: this architecture (React + FastAPI + DB + encryption) is meaningfully more complex than a single Streamlit app. It is achievable solo in 12 hours only if the encryption and DB layers are kept intentionally minimal (see Section 6) — do not let security polish crowd out the 6 mandatory requirements in Section 1.

4. Database Management & Centralized API Structure

Rather than processing the CSV inline inside route handlers, the backend is organized in clear, centralized layers so each part has one responsibility — this is the difference between a hackathon script and something that reads like production code.

Layer	File(s)	Responsibility
Config	core/config.py	Centralized settings — DB URL, secret keys, CORS origins, loaded from environment
Database	db/database.py, db/models.py	SQLAlchemy engine/session setup; ORM models (WaterSource table) — single source of truth
Schemas	schemas/water_source.py	Pydantic models for request/response validation — rejects malformed input before processing
Services	services/cleaning.py, services/stats.py, services/scoring.py	Pure business logic (Pandas/NumPy/scoring) — no HTTP or DB code, easily tested
Routers	routers/upload.py, routers/stats.py, routers/charts.py, routers/report.py	HTTP route handlers that call services, return responses; grouped by feature and endpoint
Security	core/security.py, middleware/encryption.py	Encryption/decryption helpers and middleware (see Section 6)
Entry point	main.py	Creates the FastAPI app, mounts all routers, applies middleware (CORS, encryption)

Why a database instead of just reading the CSV each time:

- Data is cleaned and validated once on ingestion, not re-parsed on every API call — faster and more consistent.
- SQLAlchemy ORM prevents raw SQL string building, which removes an entire class of SQL-injection risk.
- Filtering/searching (by area, district, status) becomes a proper indexed query instead of re-scanning a DataFrame in memory.
- A real schema (WaterSource model) documents the data contract, which is useful to show judges as evidence of production-style thinking.

For a 12-hour hackathon, SQLite is the pragmatic choice: it is a single file, needs no separate DB server to install or deploy, and SQLAlchemy makes swapping it for PostgreSQL later a one-line connection-string change if the project continues past the hackathon.

5. API Security — Encryption / Decryption & Other Improvements

Goal: every request hitting the backend is decrypted and validated before it reaches business logic, and every response is encrypted before it leaves the backend — with the React frontend performing the matching encryption/decryption so the two sides speak the same protected format.

How it works end-to-end:

- Both frontend and backend share a secret key (loaded from environment variables — .env on backend, build-time env variable on frontend — never committed to GitHub).
- Frontend (Axios interceptor): before sending a request, the JSON payload is encrypted with AES (crypto-js) into a single encrypted string field, e.g. { "payload": "U2FsdGVkX1..." }.
- Backend (FastAPI middleware): on receiving a request, middleware decrypts the payload field back into JSON before it reaches the route/service layer. Pydantic schemas then validate the decrypted data.
- Backend response: the JSON response body is encrypted the same way (AES/Fernet) before being sent back.
- Frontend (Axios response interceptor): decrypts the response payload before handing it to React components — components never see raw ciphertext.

Suggested implementation (kept lightweight for hackathon time constraints):

Side	Tool	Notes
Backend	Python cryptography library — Fernet (AES-128 Fernet symmetric MAC)	Fernet is cryptographically authenticated, and simple: one encrypt() / decrypt()
Frontend	crypto-js (AES)	Same secret key encrypts outgoing payloads and decrypts incoming
Transport	HTTPS (automatic on Vercel/Render/Railway)	This is the real-world primary protection — payload encryption is an

Honest scope note: this dataset (public water infrastructure stats) is not sensitive personal data, so payload encryption here is primarily a way to demonstrate security awareness to judges rather than a strict necessity. HTTPS transport encryption alone would already be industry-standard protection. Recommendation: implement it, but treat it as a Section 8 "if time remains" item, not something to build before the 6 mandatory requirements are done.

Other security improvements worth including:

- Environment variables for all secrets (DB path, encryption key) — .env file, added to .gitignore, never pushed to GitHub.

- CORS restricted to the exact deployed frontend origin, not a wildcard (*).
- Input validation via Pydantic schemas on every endpoint — rejects malformed/oversized payloads automatically.
- Rate limiting on upload/report endpoints (e.g. slowapi) to prevent abuse during the live demo or after deployment.
- File-type and size validation on CSV upload (reject non-CSV files, cap file size) before it ever reaches Pandas.
- Generic error responses to the client (no stack traces or internal paths leaked); detailed errors only in server-side logs.

6. Folder Structure

cleanwater-tracker/

■■■ backend/

- ■■■ main.py (*creates app, mounts routers + middleware*)
- ■■■ core/config.py (*centralized settings from .env*)
- ■■■ core/security.py (*encrypt/decrypt helper functions*)
- ■■■ middleware/encryption.py (*request decrypt / response encrypt*)
- ■■■ db/database.py, db/models.py (*SQLAlchemy engine + ORM models*)
- ■■■ schemas/water_source.py (*Pydantic request/response models*)
- ■■■ services/cleaning.py, stats.py, scoring.py, report.py
- ■■■ routers/upload.py, stats.py, charts.py, report.py
- ■■■ data/water_sources.csv (*seed data*)
- ■■■ .env (*secrets — gitignored*)
- ■■■ requirements.txt

■■■ frontend/

- ■■■ src/components/ (*UploadPanel, MetricCards, ChartsView, PriorityReport, MapView*)
- ■■■ src/api/client.js (*Axios instance + encrypt/decrypt interceptors*)
- ■■■ src/utils/crypto.js (*AES helper using crypto-js*)
- ■■■ src/App.jsx
- ■■■ .env (*API base URL + shared key — gitignored*)
- ■■■ package.json
- README.md

7. Bonus — Deployment & Submission

- Frontend deployed on Vercel (free, connects directly to GitHub repo, auto-builds React app).
- Backend deployed on Render or Railway (free tier); frontend's API base URL points to this live backend URL.
- Note: free-tier backends may 'sleep' after inactivity, causing a 30–50s delay on the first request — worth mentioning to judges before the demo, or 'waking' the backend a few minutes before presenting.
- Public GitHub repository (monorepo with /frontend and /backend) containing full source code, with .env files excluded via .gitignore.
- README with setup / run instructions for both frontend and backend (in case the tool isn't deployed).
- 2–3 minute live demo prepared for the end of the hackathon.

8. UI/UX Enhancements — React Frontend (Beyond Minimum Requirement)

- Interactive filters** — Dropdown/multiselect components (area, district, water quality status) that re-query the FastAPI backend or filter already-fetched data client-side.
- Search bar** — Debounced search input to filter by area name in real time.
- Color-coded severity** — Styled badges (Tailwind) — red / yellow / green for Unsafe / Needs Inspection / Safe — in tables and cards.
- Downloadable report** — Button that calls a backend /priority-report/download endpoint and triggers a PDF/CSV file download in-browser.
- Map view** — React map component (e.g. Leaflet/React-Leaflet) plotting source locations with markers colored by urgency.
- Summary metric cards** — Dashboard-style cards at the top — total sources tracked, unsafe count, average population affected — fetched from a /summary endpoint.
- Tabbed / routed layout** — Separate views for Overview, Charts, and Priority Report using React Router or simple tab state, for a clean single-page-app feel.
- Loading & error states** — Skeleton loaders / spinners while awaiting API responses, and graceful error messages if the backend is unreachable.
- Responsive design + branding** — Mobile-friendly layout (Tailwind grid/flex), custom header/logo, e.g. "■■■ CleanWater Access Tracker", consistent color theme and micro-animations for a modern feel.

9. AI-Enhanced Features (Extra Layer)

- Primary (guaranteed, offline-safe):** Template-based natural language generation (NLG). Python logic converts each top-priority area's structured data (name, population, status, days since inspection, complaint count) into a human-readable paragraph using conditional sentence templates — no external API, no internet dependency, cannot fail during the demo.
 - Optional enhancement (if time allows):** A toggle (e.g. checkbox: "Generate AI-enhanced summary") that sends the top-priority area data to a free LLM API — Google Gemini free tier or Groq free tier — to produce a more natural, varied narrative summary. Falls back automatically to the template-based version if the API call fails or no internet is available at demo time.
 - Basic trend forecasting (optional):** A simple NumPy-based moving average / linear trend on complaint counts over time to project whether an area's situation is worsening — adds a lightweight predictive/AI angle without needing a full ML pipeline.
- All AI/NLG logic lives in the FastAPI backend (services/report.py) and is exposed via the /priority-report endpoint; the React frontend simply renders whatever text it receives, keeping the frontend framework-agnostic to how the summary was generated.

10. Execution Priority (If Time Runs Short)

Priority	Feature
Must finish first	CSV → Pandas cleaning → DB models + ingestion → NumPy stats → 3 Matplotlib charts → Priority report
Do next	Deploy backend (Render/Railway) → deploy frontend (Vercel) → connect API URL → GitHub push → RL

Do if time remains	Encryption/decryption middleware (backend + frontend), CORS restriction, rate limiting, input validation h
Do after that	React UI enhancements (filters, map, metric cards, download button, tabs, responsive styling)
Do last / optional	AI-enhanced natural language summary via free LLM API, trend forecasting

This document consolidates the original task brief, mandatory requirements, bonus deliverables, database/backend architecture, API security (encryption/decryption), UI/UX enhancements, and AI-layer features into a single reference — intended for use as source material to generate a detailed build prompt.